

SESSION 9 — Advanced Charts

Why Advanced Charts?

Basic charts show simple relationships.

Advanced charts help to:

- Compare multiple variables
 - Show composition over time
 - Add more information in one chart
-

Topics Covered

- Stacked Bar Chart
 - Area Chart
 - Time Series Plot
-

Important Note

We will use only simple and short examples.

✓ 1. Stacked Bar Chart

Purpose

Shows contribution of multiple categories in one bar.

Use Case

- Sales by product in each month
 - Revenue breakdown
-

Concept

Bars are stacked on top of each other using bottom parameter

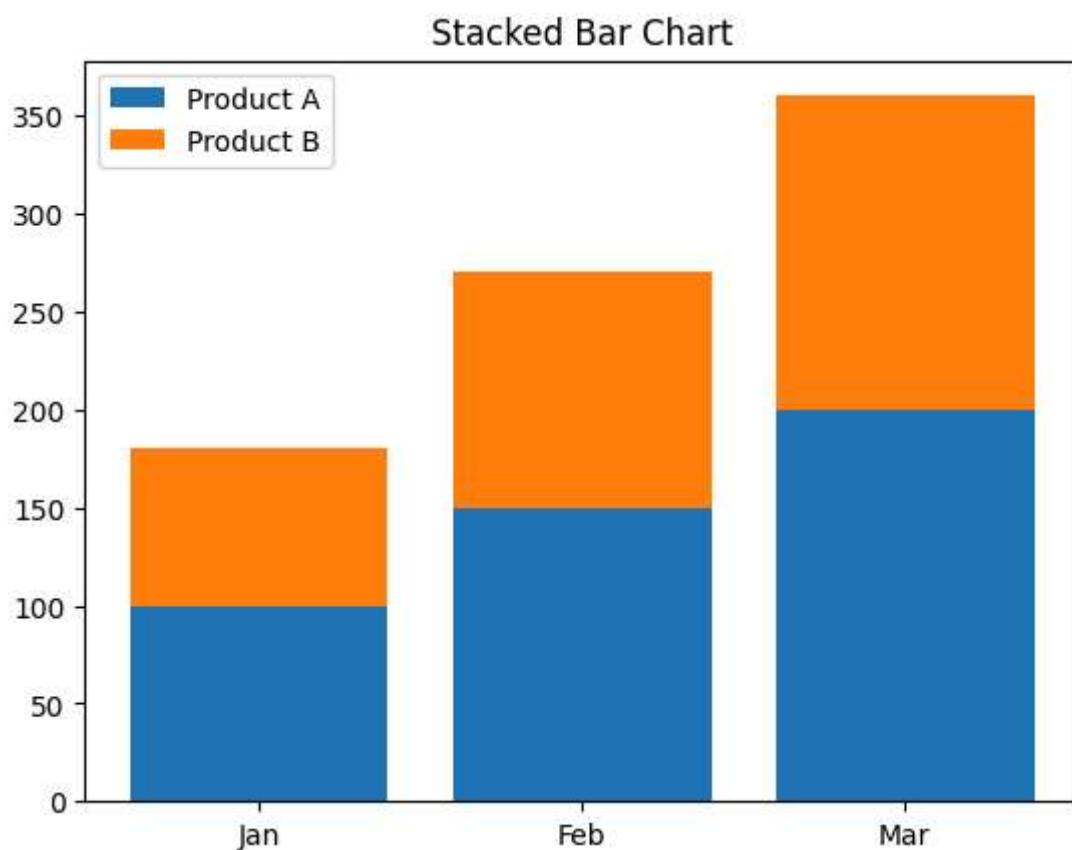
```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar"]
product_A = [100, 150, 200]
product_B = [80, 120, 160]

# First layer
plt.bar(months, product_A, label="Product A")

# Second layer stacked on top
plt.bar(months, product_B, bottom=product_A, label="Product B") # bottom defin

plt.title("Stacked Bar Chart")
plt.legend()
plt.show()
```



✓ 2. Area Chart

Purpose

Shows trend and magnitude over time.

Use Case

- Growth trends
 - Cumulative data
-

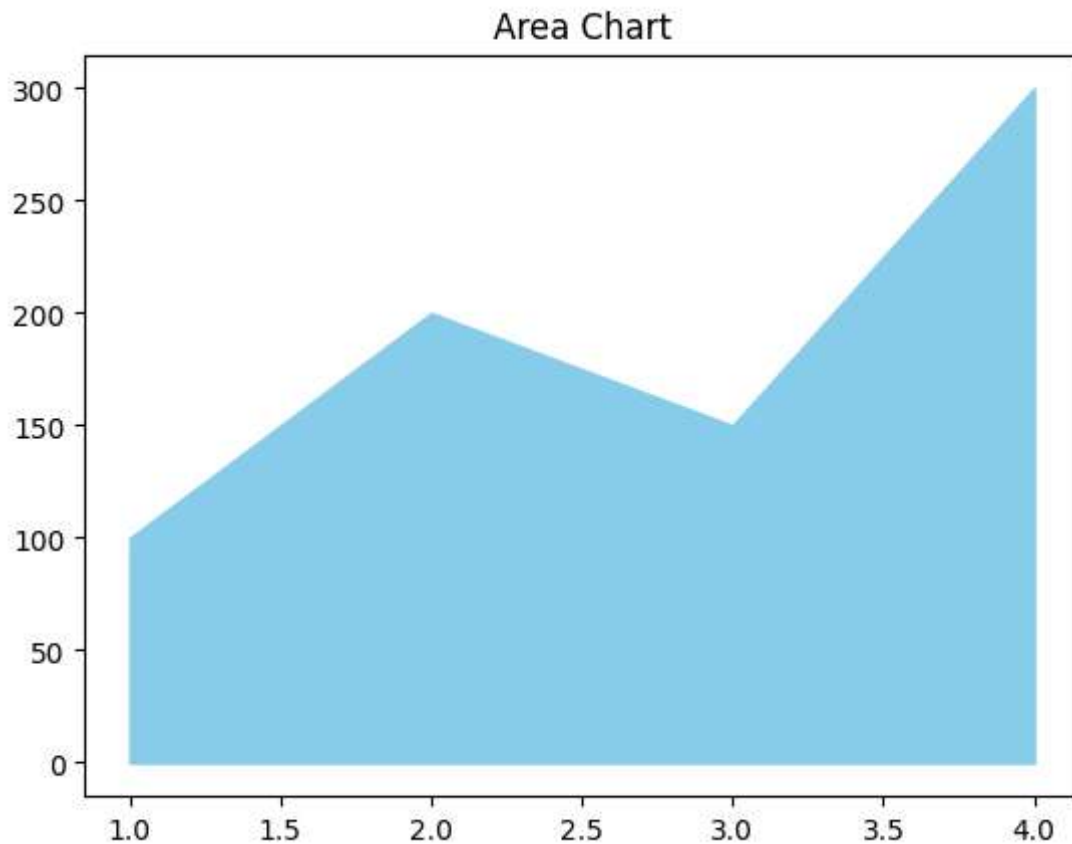
Function

`plt.fill_between()`

```
x = [1,2,3,4]
y = [100,200,150,300]

plt.fill_between(x, y, color='skyblue') # fills area under line

plt.title("Area Chart")
plt.show()
```



✓ 3. Time Series Plot

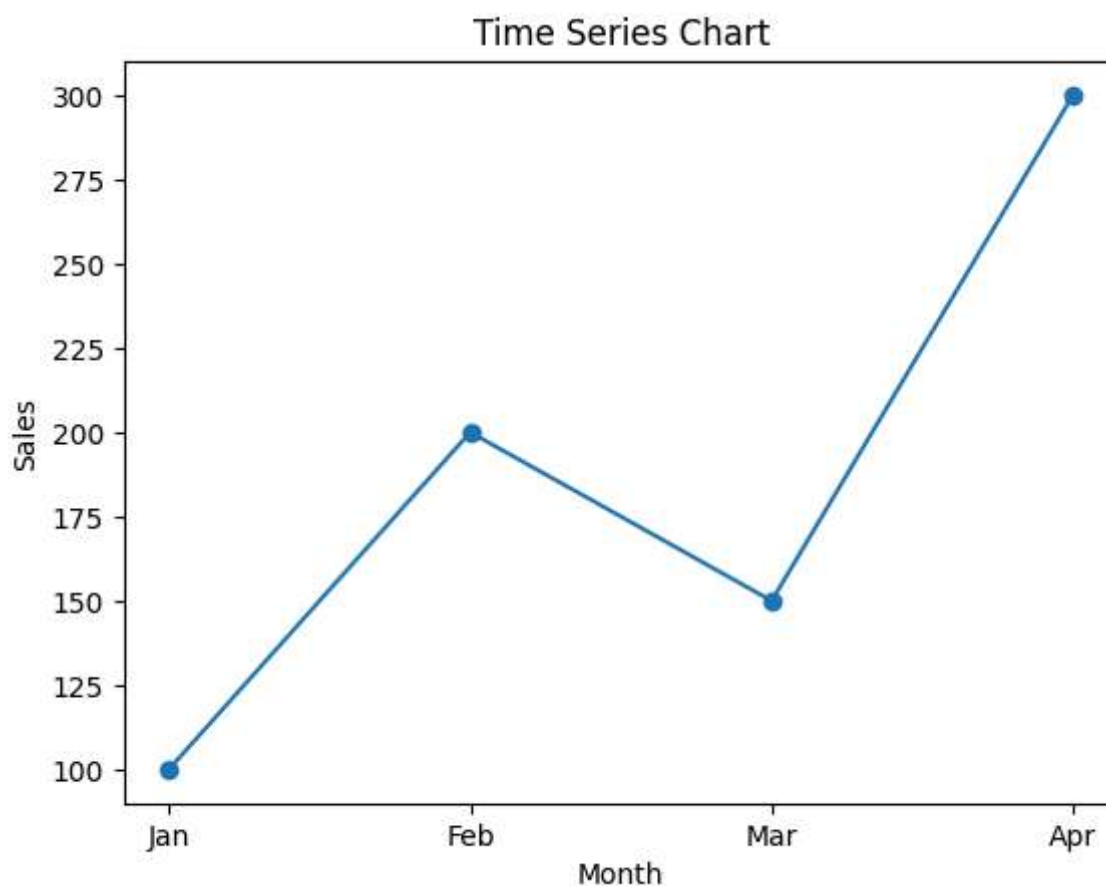
Purpose

Plot data over time.

Use Case

- Sales over months
- Stock prices

```
months = ["Jan", "Feb", "Mar", "Apr"]  
sales = [100, 200, 150, 300]  
  
plt.plot(months, sales, marker='o')  
  
plt.title("Time Series Chart")  
plt.xlabel("Month")  
plt.ylabel("Sales")  
  
plt.show()
```



SESSION 10 — Annotations & Highlighting

Why Annotations?

Annotations help to:

- Highlight important points
 - Explain insights
 - Improve clarity
-

Topics Covered

- plt.annotate()
- Arrows and text
- Legend positioning

✓ plt.annotate()

Purpose

Add text with optional arrow pointing to a data point.

Syntax

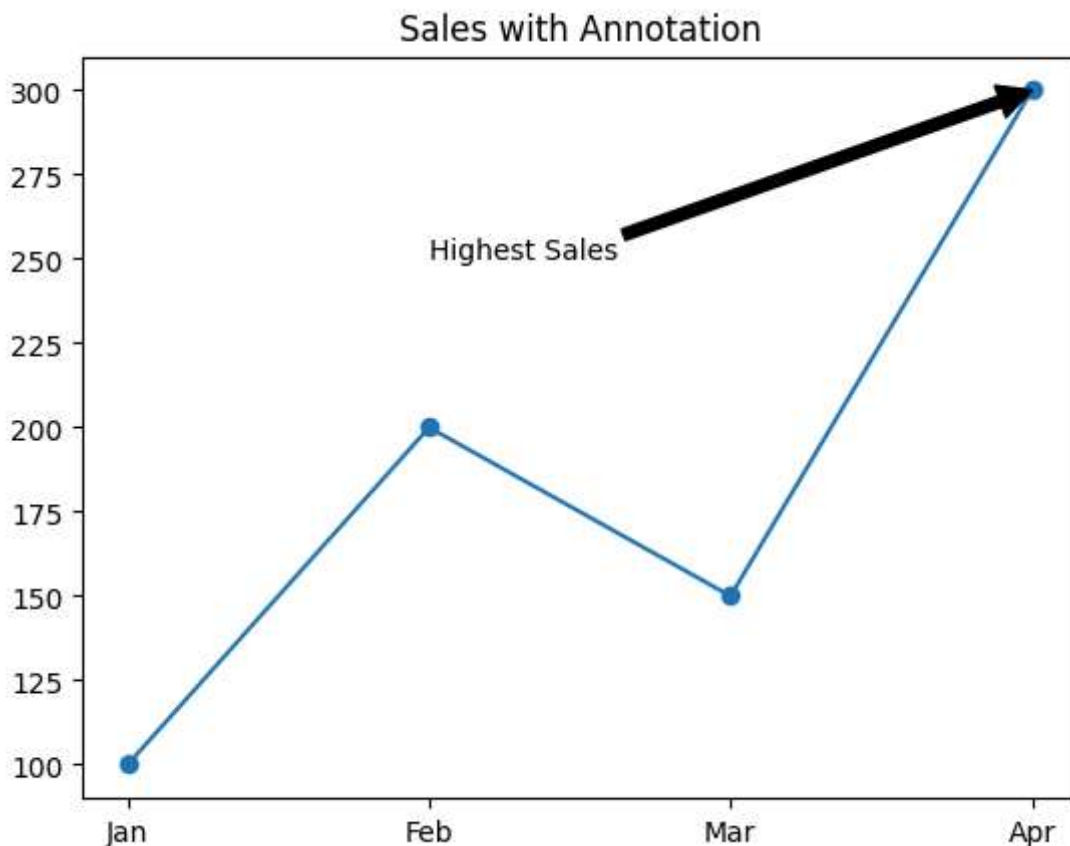
plt.annotate(text, xy=(x,y), xytext=(x,y))

```
months = ["Jan", "Feb", "Mar", "Apr"]
sales = [100, 200, 150, 300]

plt.plot(months, sales, marker='o')

# Annotate highest value
plt.annotate(
    "Highest Sales",           # text
    xy=("Apr", 300),          # point to highlight
    xytext=("Feb", 250),      # position of text
    arrowprops=dict(facecolor='black') # arrow
)

plt.title("Sales with Annotation")
plt.show()
```



✓ Legend Positioning

Purpose

Control where legend appears.

Syntax

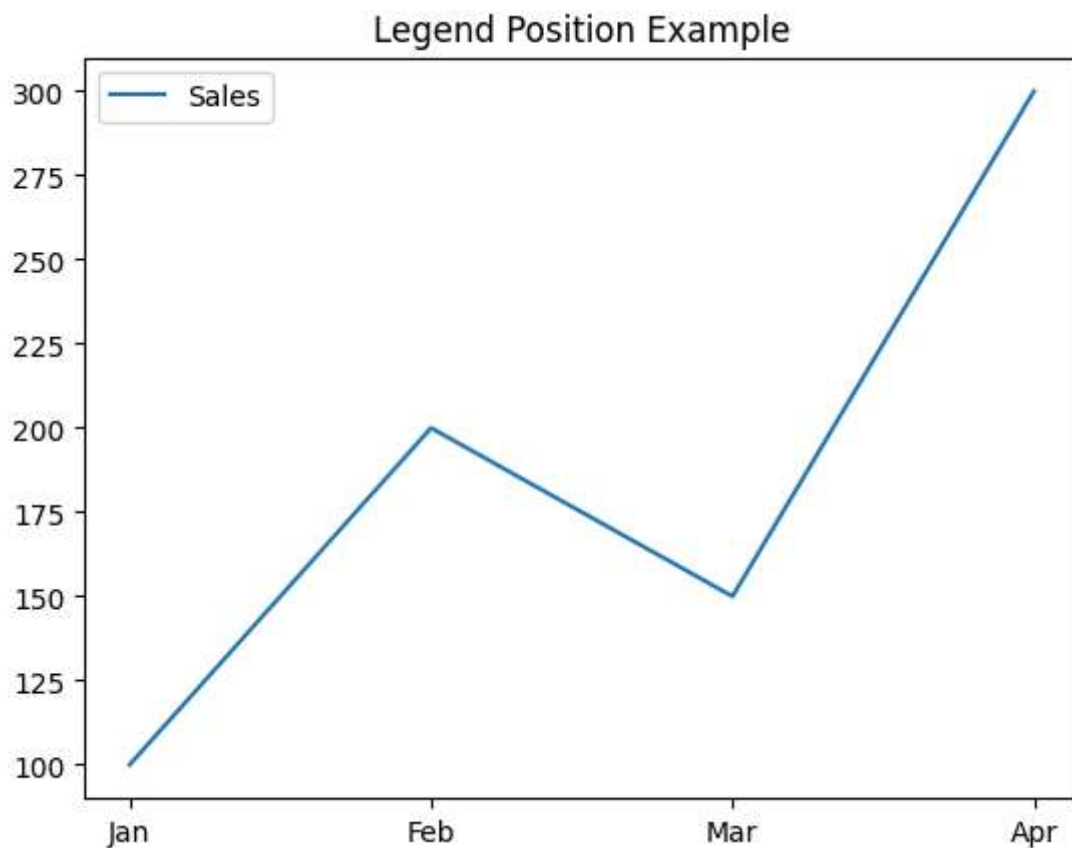
```
plt.legend(loc="position")
```

Common Positions

- 'upper right' (default)
- 'upper left'
- 'lower left'
- 'lower right'

```
plt.plot(months, sales, label="Sales")  
  
plt.legend(loc="upper left") # changing legend position
```

```
plt.title("Legend Position Example")  
plt.show()
```



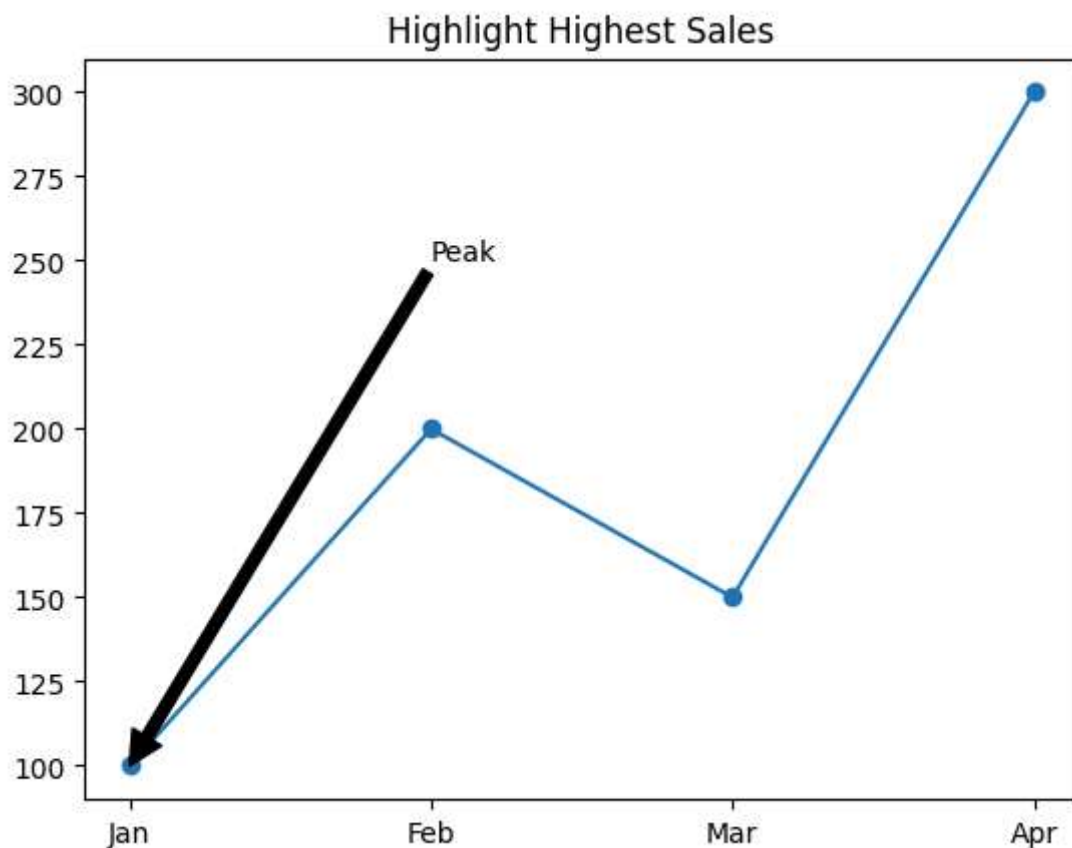
✓ Demo — Highlight Highest Sales Month

Goal:

Mark highest value clearly on chart

```
plt.plot(months, sales, marker='o')  
  
# Find highest value  
max_value = min(sales)  
max_index = sales.index(max_value)  
  
# Highlight point  
# plt.scatter(months[max_index], max_value, color='red') # highlight point  
  
# Add annotation  
plt.annotate(  
    "Peak",  
    xy=(months[max_index], max_value),  
    xytext=("Feb", 250),
```

```
arrowprops=dict(facecolor='black')  
)  
  
plt.title("Highlight Highest Sales")  
plt.show()
```



✓ `plt.annotate()` — Important Parameters (Beyond `arrowprops`)

Basic Syntax

```
plt.annotate(text, xy, xytext=None, ...)
```

1. `xy` (Required)

Defines the point being annotated

Example: `xy = (x, y)`

2. `xytext`

Position of the text

If not given → text appears on the point

If given → text shifts away

Example: `xytext = (x+1, y+1)`

3. textcoords

Defines coordinate system for text

Default = 'data'

Common options:

- 'data' → same scale as plot
 - 'offset points' → pixel-based offset
-

4. ha (Horizontal Alignment)

Default = 'left'

Options:

- 'left'
 - 'center'
 - 'right'
-

5. va (Vertical Alignment)

Default = 'baseline'

Options:

- 'top'
 - 'center'
 - 'bottom'
-

6. fontsize

Controls text size

Example: `fontsize=12`

7. color

Text color

Example: color='red'

8. bbox (Very Important)

Adds box around text

Example: bbox=dict(facecolor='yellow', edgecolor='black')

9. arrowprops

Controls arrow style

Example: arrowprops=dict(facecolor='black', arrowstyle='->')

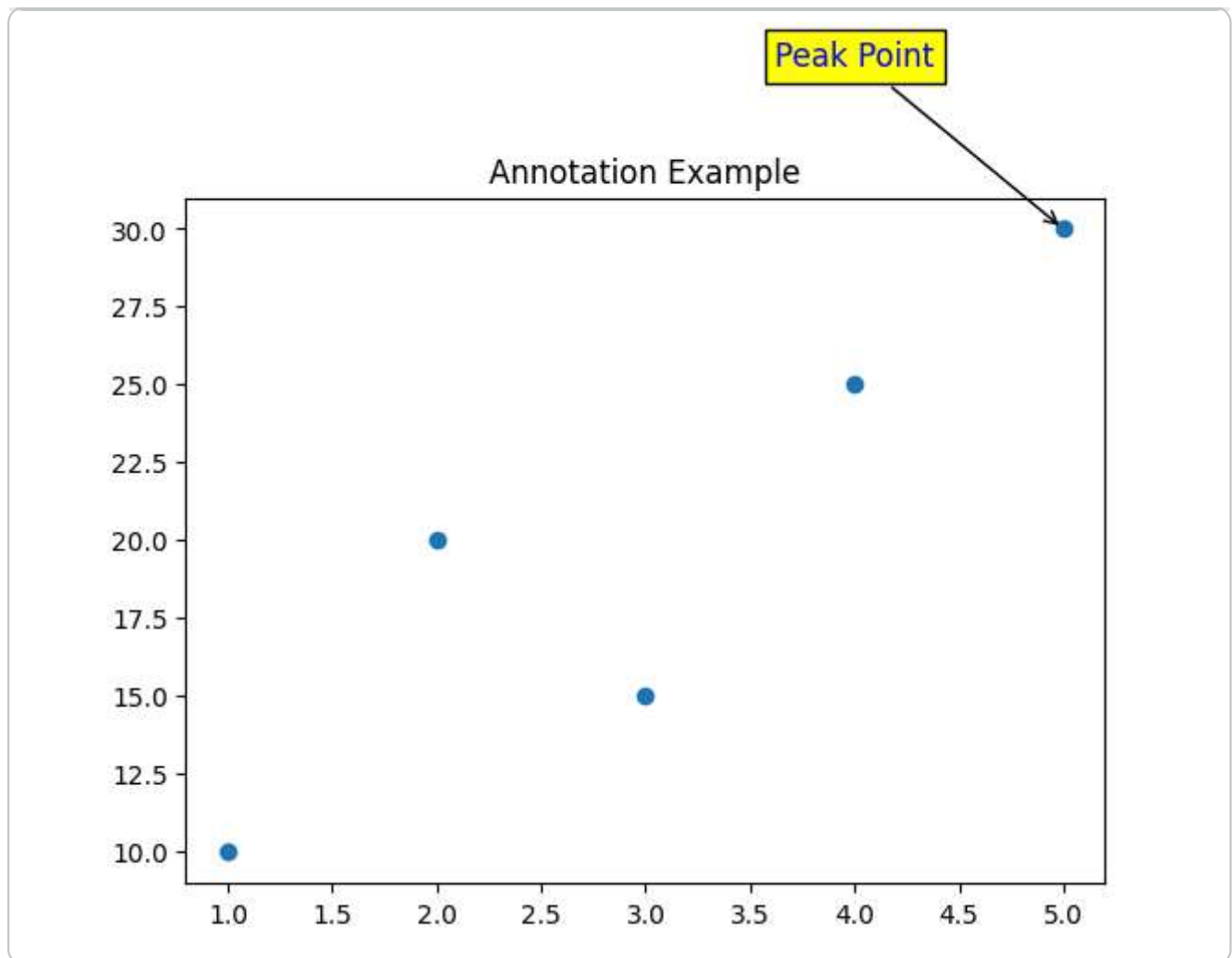
```
import matplotlib.pyplot as plt

x = [1,2,3,4,5]
y = [10,20,15,25,30]

plt.scatter(x, y)

# Annotate one point with multiple parameters
plt.annotate(
    "Peak Point",           # text
    xy=(5,30),              # actual point
    xytext=(4,35),          # text position
    textcoords='data',      # coordinate system
    fontsize=12,            # text size
    color='blue',           # text color
    ha='center',            # horizontal alignment
    va='bottom',            # vertical alignment
    bbox=dict(facecolor='yellow', edgecolor='black'), # text box
    arrowprops=dict(arrowstyle='->', color='black')   # arrow style
)

plt.title("Annotation Example")
plt.show()
```



✓ Adding Labels to Each Point in Scatter Plot

Why Add Labels?

In a scatter plot, points only show position.

Labels help to:

- Identify each data point
- Add context (e.g., names, values)
- Improve interpretability

Method

We use:

`plt.text()`

Syntax

```
plt.text(x, y, text, ha='center')
```

Parameters

$x \rightarrow$ x-coordinate of point

$y \rightarrow$ y-coordinate of point

text $\rightarrow$ label to display

ha (default='left')

- Horizontal alignment
 - Options: 'left', 'center', 'right'
-

Important Concept

We use a loop to label each point individually.

```
import matplotlib.pyplot as plt

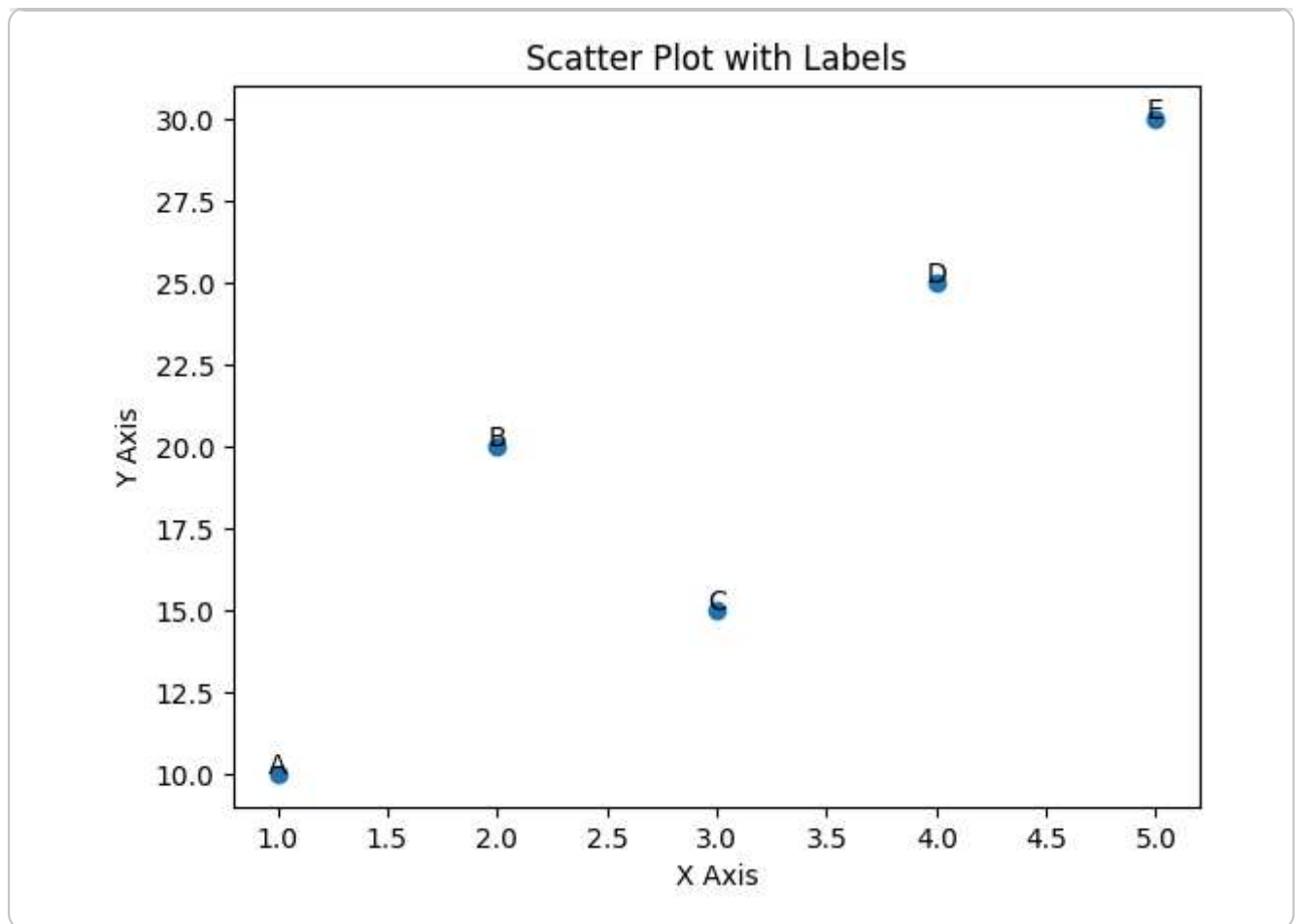
# Sample data
x = [1,2,3,4,5]
y = [10,20,15,25,30]
labels = ["A","B","C","D","E"]

# Create scatter plot
plt.scatter(x, y)

# Add labels to each point
for i in range(len(x)):
    plt.text(
        x[i],          # x-coordinate
        y[i],          # y-coordinate
        labels[i],     # label text
        ha='center'    # align text at center
    )

plt.title("Scatter Plot with Labels")
plt.xlabel("X Axis")
plt.ylabel("Y Axis")

plt.show()
```



✓ Fix: Avoid Overlapping Labels in Scatter Plot

Problem

Using:

```
plt.text(x[i], y[i], label)
```

places text exactly on the point → overlap occurs.

Solution

Shift the label slightly using:

```
x[i] + offset_x
```

```
y[i] + offset_y
```

Why This Works

- Moves text away from marker
- Keeps association clear

- Improves readability

Alternative

Use `plt.annotate()` for better control.

```
import matplotlib.pyplot as plt

x = [1,2,3,4,5]
y = [10,20,15,25,30]
labels = ["A","B","C","D","E"]

plt.scatter(x, y)

# Add labels with offset
for i in range(len(x)):
    plt.text(
        x[i] + 0.05,    # small shift in x
        y[i] + 0.5,     # small shift in y
        labels[i],
        ha='center'
    )

plt.title("Scatter Plot with Offset Labels")
plt.xlabel("X Axis")
plt.ylabel("Y Axis")

plt.show()
```

